



Anahtar Kavramlar

Optimizasyon & Çarpışma	
Kavram	Tanım
Profiling	cProfile/pstats ile darboğaz tespiti
Kare bütçesi	60 FPS=16.67 ms
Hot path	Zamanın %80'ini yiyen kod yolu
Culling	Ekran dışı sprite atlama
Object pool	Yeniden kullanım, GC azaltma
Geniş faz	Hızlı eleme (broad-phase)
Dar faz	Kesin geometri testi (narrow)
AABB	Eksen hizalı kuşatan kutu
Spatial hash	Sabit hücre indeksleme
Quad tree	Özyinelemeli alt bölme

AI & Dağıtım	
Kavram	Tanım
FSM	Sonlu durum makinesi
A*	$f = g + h$ heuristik yol bulma
Heuristic	Hedef tahmini mesafe
Steering	Vektör tabanlı manevra
PyInstaller	Python → exe paketleyici
_MEIPASS	Paketlenmiş varlık yolu
itch.io	İndie oyun pazarı
butler	itch.io CLI yükleyici

Kare Bütçesi Tablosu

Hedef FPS		
FPS	Bütçe / Kullanım	
30	33.33 ms / mobil	
60	16.67 ms / standart	
120	8.33 ms / kompetitif	
144	6.94 ms / üst seviye	

Profiling Komutları

Manuel Ölçüm	
<pre>import time t0 = time.perf_counter() # olculecek kod t1 = time.perf_counter() print(f"{{(t1-t0)*1000:.2f}} ms")</pre>	

cProfile + pstats	
<pre>import cProfile, pstats prof = cProfile.Profile() prof.enable() main() prof.disable() st = pstats.Stats(prof) st.sort_stats('cumulative') st.print_stats(15)</pre>	

pstats Sıralama	
Anahtar	Anlam
cumulative	alt çağrılar dahil
tottime	sadece o fonksiyon
ncalls	toplam çağrı sayısı
percall	çağrı başına süre

Culling

Ekran Dışı Atlama	
<pre>ekran_rect = ekran.get_rect() for s in tum_sprite: if ekran_rect.colliderect(s.rect): ekran.blit(s.image, s.rect)</pre>	

Object Pool

Generic Pool	
<pre>class Pool: def __init__(self, sinif, k): self.kullanilabilir = [sinif() for _ in range(k)] self.aktif = [] def acquire(self): if not self.kullanilabilir: return None n = self.kullanilabilir.pop() self.aktif.append(n) return n def release(self, n): if n in self.aktif: self.aktif.remove(n) self.kullanilabilir.append(n)</pre>	

Pool'a Uyumlu Mermi	
<pre>class Mermi(Sprite): def __init__(self): super().__init__() self.image = Surface((6,12), SRCALPHA) self.rect = self.image.get_rect() self.active = False def firlat(self, x, y, vy=-9): self.rect.center = (x, y) self.vy = vy self.active = True</pre>	

Geniş / Dar Faz

İki Aşamalı Çarpışma	
Faz	Amaç
Geniş	hızlı ele, AABB testi
Dar	kesin geometri (mask vb.)

AABB Testi	
<pre>def aabb(a, b): return (a.right > b.left and a.left < b.right and a.bottom > b.top and a.top < b.bottom)</pre>	

Spatial Hash

Hücre Tabanlı İndeks	
<pre>from collections import defaultdict class SpatialHash: def __init__(self, cell): self.cells = defaultdict(list) self.cell = cell def _h(self, r): x0 = r.left // self.cell y0 = r.top // self.cell x1 = r.right // self.cell y1 = r.bottom // self.cell return [(cx, cy) for cx in range(x0, x1+1) for cy in range(y0, y1+1)] def insert(self, o): for k in self._h(o.rect): self.cells[k].append(o) def query(self, r): sonuc = set() for k in self._h(r): sonuc.update(self.cells[k]) return sonuc</pre>	

Hücre Boyutu Seçimi	
Hücre	Etki
Çok küçük	çok hücre, çakışma
Sprite x2	ideal ortalama
Çok büyük	hücre başına çok nesne

Quad Tree

Özyinelemeli Bölme	
<pre>class QuadTree: KAPASITE = 4 DERINLIK_MAX = 6 def __init__(self, sinir, d=0): self.sinir = sinir self.d = d self.nesneler = [] self.alt = None def _bol(self): x, y = self.sinir.topleft w, h = (self.sinir.w//2, self.sinir.h//2) self.alt = [QuadTree(Rect(x, y, w,h), self.d+1), QuadTree(Rect(x+w, y, w,h), self.d+1), QuadTree(Rect(x, y+h, w,h), self.d+1), QuadTree(Rect(x+w, y+h, w,h), self.d+1),]</pre>	

Karşılaştırma	
Yaklaşım	Karmaşıklık
Brute force	$O(n^2)$
Spatial hash	$O(n)$ ortalama
Quad tree	$O(n \log n)$
Sprite grup	$O(n^2) + C$

FSM (Karakter)

Düşman FSM Şablonu	
<pre>class DusmanFSM: def __init__(self): self.durum = "idle" def guncelle(self, dt, hedef): if self.durum == "idle": if self._gorus(hedef): self.durum = "chase" elif self.durum == "chase": self._takip(hedef) if self._yakin(hedef): self.durum = "attack" elif self.durum == "attack": self._saldir() if not self._yakin(hedef): self.durum = "chase"</pre>	

Tipik Durumlar	
Durum	Davranış
idle	beklemek
patrol	noktalar arası
chase	oyuncuyu izle
attack	saldırı uygula
flee	uzaklaş

A* Yol Bulma

Manhattan Heuristik	
<pre>def h(a, b): return (abs(a[0]-b[0]) + abs(a[1]-b[1]))</pre>	

A* Çekirdeği	
<pre>import heapq def astar(grid, baslangic, hedef): acik = [(0, baslangic)] g = {baslangic: 0} geldigi = {} while acik: _, u = heapq.heappop(acik) if u == hedef: return _yol(geldigi, u) for v in komsular(grid, u): yeni = g[u] + 1 if yeni < g.get(v, 1e9): g[v] = yeni f = yeni + h(v, hedef) heapq.heappush(acik, (f, v)) geldigi[v] = u return []</pre>	

A* Anahtar Kavramlar	
Sembol	Anlam
g	başlangıçtan gerçek maliyet
h	hedefe tahmini maliyet
f = g+h	toplam tahmin
heapq	öncelikli kuyruk

Steering Behaviors

Seek (Hedefe Git)	
<pre>def seek(konum, hedef, hiz): yon = (hedef - konum) if yon.length() == 0: return Vector2(0, 0) return yon.normalize() * hiz</pre>	
Flee & Arrival	
<pre>def flee(konum, tehlike, hiz): return -seek(konum, tehlike, hiz) def arrival(konum, hedef, hiz, yavas_r=80): yon = hedef - konum d = yon.length() if d == 0: return Vector2() olcek = min(1, d / yavas_r) return yon.normalize() * hiz * olcek</pre>	

PyInstaller

Temel Komut	
<pre>pyinstaller \ --onefile \ --windowed \ --name UzayKacisi \ --add-data "assets:assets" \ --icon=icon.ico \ oyun.py</pre>	

Bayraklar	
Bayrak	Etki
-onefile	tek exe
-windowed	terminal yok
-name	çıkı adı
-add-data	varlık ekle
-icon	simge
-exclude-module	boyut küçült

Varlık Yönetimi

resource_path	
<pre>import sys, os def resource_path(yol): if hasattr(sys, "_MEIPASS"): kok = sys._MEIPASS else: kok = os.path.abspath(".") return os.path.join(kok, yol) # Kullanım: img = pygame.image.load(resource_path("assets/oyuncu.png"))</pre>	

Add-Data Sözdizimi	
İS	Ayraç
macOS / Linux	kaynak:hedef
Windows	kaynak;hedef

itch.io & butler

butler ile Yükleme	
<pre>butler login butler push UzayKacisi.zip \ kullanıcı/uzay-kacisi:windows butler status \ kullanıcı/uzay-kacisi</pre>	

Yayın Akışı	
- PyInstaller ile paketle - Klasörü test et - ZIP olarak sıkıştır - itch.io'da sayfa aç - butler push ile yükle - Sürüm notu yaz	

Yaygın Hatalar

Optimizasyon Hataları		
Belirti		Çözüm
FPS düşük, neden bilinmiyor		cProfile ile ölç
Mermi spam'inde lag		object pool
Çok sprite'ta donma		spatial hash
Ekran dışı çizim		culling rect
GC duraklamaları		pool + yeniden kullan
Tek karede stutter		16.67 ms aşıldı
Erken optimizasyon		önce ölç sonra düzelt

Çarpışma & AI Hataları		
Belirti		Çözüm
Düşman bilet gibi		FSM ekle
Engelli haritada takılma		A* path
A* çok yavaş		grid çözünürlük düşür
Steering tireşim		arrival yavaşlat
Quad tree boş düğüm		kapasite/derinlik ayarla
Hash hücresi seçimi		sprite x 2

Dağıtım Hataları	
Belirti	Çözüm
Assets bulunamıyor	resource_path + add-data
exe çok büyük	-exclude-module
Konsol açılıyor	-windowed
Antivirüs uyarısı	imzalı sertifika
itch.io yükleme yavaş	butler kullan
Yol ayraç hatası	İS'ye göre : ya da ;

Çapraz Referanslar

Önceki Bölümlerde	
Bölüm	Bağlantı
B6 çarpışma	B13 Ünite 2
B8 animasyon	B13 Ü1 kare bütçesi
B10 tile	B13 Ü3 A* grid
B11 düşmanlar	B13 Ü3 FSM/AI
B12 oyun-state	B13 Ü3 karakter-state
B12 dosya	B13 Ü4 .spec dosyası

Altın Kural
1. Önce ölç (cProfile).
2. Hot path 'i bul.
3. Sadece o noktayı düzelt.
4. Yeniden ölç ve karşılaştır.
5. Yayınlamadan önce paketi <i>başka bilgisayarda</i> test et.